



МИНИСТЕРСТВО СЕЛЬСКОГО ХОЗЯЙСТВА РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«РОССИЙСКИЙ ГОСУДАРСТВЕННЫЙ АГРАРНЫЙ УНИВЕРСИТЕТ – МСХА имени
К. А. ТИМИРЯЗЕВА»
(ФГБОУ ВО РГАУ – МСХА имени К. А. Тимирязева)

Институт экономики и управления
Кафедра статистики и кибернетики
Направление: 09.03.02 Информационные системы и технологии
Направленность: Большие данные и машинное обучение
Методы и средства проектирования информационных систем и технологий

КУРСОВОЙ ПРОЕКТ

на тему: «Проектирование и разработка веб-приложения «Цифровой агроном» с
ИИ-ассистентом для поддержки принятия решений в растениеводстве».

Выполнил:

Студент 3 курса группы Д-Э-311

Бондарев И. А.

Дата регистрации КП

на кафедре: _____

Допущен к защите

Руководитель:

учёная степень, учёное звание, ФИО

Члены комиссии:

учёная степень, учёное звание, ФИО подпись

учёная степень, учёное звание, ФИО подпись

учёная степень, учёное звание, ФИО подпись

Оценка _____

Дата защиты _____

Москва 2026

Содержание

Введение	3
Глава 1. Теоретическая часть	5
1.1. Эволюция систем поддержки принятия решений в сельском хозяйстве	5
1.2. Методы искусственного интеллекта и машинного обучения в растениеводстве	7
1.3. Проблемы импортозамещения в области ИТ для сельского хозяйства	9
1.4. Обоснование выбора технологического стека	10
1.5. Критерии оценки качества системы	12
1.6. Выводы по первой главе	14
Глава 2. Практическая часть	15
2.1. Архитектура веб-приложения	15
2.2. Реализация серверной части	16
2.3. Реализация ML-модели прогнозирования урожайности и сравнения культур	19
2.4. Реализация ИИ-ассистента и клиентской части	21
2.5. Тестирование системы и выводы по второй главе	23
Заключение	25
Список использованных источников	26
Приложение	28
Листинг Б.1 — app.py	32
Листинг Б.2 — crop_comparator.py	34
Листинг Б.3 — rotation_predictor.py	36
Листинг Б.4 — ai_assistant.py	38
Листинг Б.5 — ml_analytics.py	40
Листинг Б.6 — config.py	42
Листинг Б.7 — index.html	43
Листинг Б.8 — chat.html	46

Введение

Современное сельское хозяйство переживает период цифровой трансформации, которая затрагивает все аспекты агропромышленного комплекса — от управления фермерскими хозяйствами до прогнозирования урожайности и диагностики болезней растений. Внедрение информационных технологий в аграрный сектор позволяет существенно повысить эффективность производства, снизить потери урожая и оптимизировать использование ресурсов.

Особую актуальность в последние годы приобретают системы поддержки принятия решений на основе искусственного интеллекта. Такие системы способны анализировать большие объёмы данных, выявлять скрытые закономерности и предоставлять рекомендации, сравнимые по качеству с советами опытных агрономов. Это особенно важно для малых и средних фермерских хозяйств, которые не всегда могут позволить себе нанять квалифицированного агронома.

Однако распространение зарубежных решений в России столкнулось с серьёзными препятствиями. Начиная с 2022 года, многие международные ИТ-компании прекратили свою деятельность на территории РФ, а доступ к ключевым сервисам разработки (PyPI, GitHub, Hugging Face) был ограничен. Это создало необходимость поиска альтернативных путей разработки и использования технологий искусственного интеллекта в российском аграрном секторе.

В связи с этим разработка отечественного веб-приложения «Цифровой агроном», которое объединяет возможности машинного обучения для прогнозирования урожайности и консультационного ИИ-ассистента, является своевременной и востребованной задачей. Такое приложение может стать доступным инструментом для фермеров, агрономов и садоводов, предоставляя им квалифицированную помощь в режиме 24/7.

Целью данной работы является проектирование, разработка и экспериментальная апробация веб-приложения «Цифровой агроном» с ИИ-ассистентом, обеспечивающего прогнозирование урожайности на основе ML-модели и консультационную поддержку.

Для достижения цели требуется решить следующие задачи:

- 1) Провести анализ существующих систем поддержки принятия решений в

сельском хозяйстве, выявить их достоинства и недостатки.

- 2) Изучить проблемы импортозамещения и доступности технологий искусственного интеллекта на территории Российской Федерации.
- 3) Обосновать выбор оптимального стека технологий с учётом выявленных ограничений.
- 4) Спроектировать модульную архитектуру веб-приложения, определить взаимодействие между компонентами.
- 5) Реализовать программный прототип, включающий ML-модель прогнозирования урожайности и ИИ-ассистента.
- 6) Провести тестирование разработанной системы и оценить качество её работы по различным метрикам.

Объектом исследования являются процессы поддержки принятия решений в растениеводстве, включая прогнозирование урожайности, выбор культур для посева и консультационную поддержку.

Предметом исследования выступают методы машинного обучения (в частности, Random Forest) для прогнозирования урожайности, а также технологии построения ИИ-ассистентов на основе больших языковых моделей и API-сервисов.

Практическая значимость работы состоит в возможности использования разработанного веб-приложения фермерскими хозяйствами, агрономическими службами и садоводами для поддержки принятия решений по выбору культур, планированию севооборота и получению консультаций по уходу за растениями.

Методология исследования включает анализ научной и технической литературы, изучение документации библиотек машинного обучения, проектирование архитектуры приложения, реализацию программного прототипа на языке Python с использованием фреймворка Flask и библиотеки scikit-learn, а также экспериментальную оценку качества работы системы.

Глава 1. Теоретическая часть

1.1. Эволюция систем поддержки принятия решений в сельском хозяйстве

Системы поддержки принятия решений (СППР) в сельском хозяйстве прошли долгий путь эволюции — от простых бумажных справочников до сложных интеллектуальных систем на основе искусственного интеллекта [13]. Понимание этой эволюции важно для оценки текущего уровня развития технологий и определения направлений их совершенствования.

На начальном этапе (1960–1980-е годы) основным инструментом агронома были бумажные карты полей, справочники по сортам культур, удобрениям и средствам защиты растений. Агрономы полагались исключительно на личный опыт и интуицию, что приводило к высокому риску ошибок и невозможности масштабирования знаний. Каждое хозяйство накапливало собственный опыт, который часто не передавался другим.

С появлением персональных компьютеров в 1980-х годах начали разрабатываться первые электронные базы данных по сортам культур, удобрениям и средствам защиты растений. Это позволило ускорить доступ к информации, систематизировать справочные данные и упростить их поиск. Однако такие системы были пассивными — они предоставляли информацию, но не анализировали её и не генерировали рекомендации.

В 1990-е годы с развитием спутниковых технологий и глобальных систем позиционирования (GPS) возникло понятие «точного земледелия» (precision agriculture) [12]. Появились системы мониторинга полей, позволяющие учитывать пространственную неоднородность почв и рассчитывать вегетационный индекс растений (NDVI) на основе спутниковых снимков. Эти системы собирали большие объёмы пространственных данных, но их анализ всё ещё требовал участия человека. Именно в этот период начали формироваться предпосылки для использования методов машинного обучения в сельском хозяйстве.

Современный этап (2010-е годы – настоящее время) характеризуется массовым внедрением технологий искусственного интеллекта, машинного обучения и компьютерного зрения. Этот переход стал возможным благодаря трём ключевым факторам: росту вычислительных мощностей (особенно

графических процессоров GPU), доступности больших размеченных датасетов (например, PlantVillage содержит более 54 тысяч изображений здоровых и больных растений) и развитию библиотек глубокого обучения (TensorFlow, PyTorch, Keras).

В настоящее время существует несколько категорий систем поддержки принятия решений в сельском хозяйстве:

- Диагностика болезней по фото (Plantix) — мобильное приложение, использующее свёрточные нейронные сети для определения болезней растений по фотографии. Охватывает более 30 культур и 400 болезней. Пользователю достаточно сфотографировать лист растения, и приложение в течение нескольких секунд выдаёт диагноз и рекомендации по лечению.
- Мониторинг полей (Climate FieldView) — комплексная платформа, объединяющая спутниковые снимки, данные с наземных датчиков, историю полей и погодные сервисы. Система позволяет оптимизировать сроки посева, полива и внесения удобрений.
- Экспертные системы (Agrobase) — базы знаний по средствам защиты растений с модулем принятия решений. Агроном может указать культуру, стадию вегетации и наблюдаемые симптомы, а система предложит список рекомендуемых препаратов.
- ERP-системы для АПК — комплексные системы управления сельскохозяйственным предприятием, включающие модули планирования посевных площадей, учёта материальных ресурсов, анализа эффективности производства.
- ИИ-ассистенты — чат-боты на основе больших языковых моделей, отвечающие на вопросы фермеров в диалоговом режиме. Такие системы могут давать советы по уходу за культурами, диагностировать болезни по описанию симптомов и интерпретировать результаты прогнозов.

Анализ существующих решений позволяет сделать следующие выводы. Во-первых, большинство зарубежных продуктов (Plantix, Climate FieldView) после 2022 года стали недоступны для российских пользователей или требуют оплаты в валюте, что затрудняет их использование. Во-вторых, существующие

экспертные системы часто содержат устаревшую информацию, так как обновление баз знаний требует постоянных усилий разработчиков. В-третьих, универсальные ИИ-ассистенты (ChatGPT, GigaChat, YandexGPT) не обладают специализированными знаниями в области агрономии и могут давать общие или неточные ответы на профессиональные вопросы.

Таким образом, актуальность разработки отечественной системы поддержки принятия решений в растениеводстве не вызывает сомнений. Такая система должна объединять возможности машинного обучения для прогнозирования, экспертные знания по севообороту и современные технологии диалоговых ИИ-ассистентов.

1.2. Методы искусственного интеллекта и машинного обучения в растениеводстве

Современные методы искусственного интеллекта открывают новые возможности для автоматизации диагностики, прогнозирования и консультирования в сельском хозяйстве. Рассмотрим основные группы методов.

Прогнозирование урожайности. Это задача регрессии — предсказания непрерывного значения на основе набора признаков: климатических (температура, осадки, влажность, солнечная радиация), почвенных (тип почвы, pH, содержание гумуса), агротехнических (культура, сорт, удобрения, пестициды) и временных (год, сезон, фаза вегетации).

Алгоритм Random Forest (случайный лес) [10] является одним из наиболее эффективных методов для прогнозирования урожайности. Он представляет собой ансамбль (совокупность) множества деревьев решений, каждое из которых обучается на случайной подвыборке данных. Итоговый прогноз получается усреднением предсказаний всех деревьев. Ключевые преимущества Random Forest для данной задачи: устойчивость к переобучению благодаря усреднению большого количества деревьев; способность работать с разнородными признаками (категориальными и числовыми) без предварительного масштабирования; автоматическое определение важности признаков — можно понять, какие факторы наиболее сильно влияют на урожайность; устойчивость к выбросам и пропускам в данных.

Другим популярным методом является Gradient Boosting (градиентный бустинг), реализованный в библиотеках XGBoost, LightGBM и CatBoost. В отличие от Random Forest, где деревья строятся независимо, в градиентном бустинге каждое следующее дерево обучается на ошибках предыдущего. Это позволяет достичь более высокой точности, но требует больше вычислительных ресурсов и более тщательной настройки гиперпараметров.

Для учёта временных зависимостей в данных (например, последовательности урожайности по годам) применяются рекуррентные нейронные сети, в частности архитектура LSTM (Long Short-Term Memory) [14]. LSTM способна «запоминать» долгосрочные зависимости, что важно для прогнозирования по временным рядам. Однако нейросетевые методы требуют значительно большего объёма данных и вычислительных ресурсов по сравнению с Random Forest.

Компьютерное зрение. Диагностика болезней по фотографиям листьев является задачей классификации изображений. Традиционные методы основаны на выделении признаков вручную (цветовые гистограммы, текстурные дескрипторы) с последующей классификацией (SVM, Random Forest). Методы глубокого обучения используют свёрточные нейронные сети (CNN), которые автоматически извлекают признаки. Наиболее популярные архитектуры: ResNet50 (точность 99,2%), InceptionV3 (98,5%), YOLOv8 (97,7%) [15]. Основное преимущество CNN — инвариантность к масштабу, повороту и сдвигу изображения.

Большие языковые модели (LLM). LLM — нейросетевые архитектуры на основе трансформеров с механизмом внимания (attention) [11], обученные на огромных объёмах текстовых данных (триллионы токенов). Ключевая инновация — механизм внимания, позволяющий модели фокусироваться на наиболее релевантных частях входного текста при генерации ответа.

Применение LLM в сельском хозяйстве: консультации по уходу за культурами в естественном диалоговом режиме; диагностика болезней по текстовому описанию симптомов; рекомендации по лечению и профилактике; интерпретация ML-прогнозов для конечного пользователя.

Архитектура RAG (Retrieval-Augmented Generation) дополняет LLM поиском по внешней базе знаний, что позволяет использовать актуальные агрономические справочники и снижает риск «галлюцинаций» модели.

1.3. Проблемы импортозамещения в области ИТ для сельского хозяйства

Начиная с 2022 года, российские разработчики столкнулись с серьёзными ограничениями доступа к зарубежной ИТ-инфраструктуре [4]. Рассмотрим основные проблемы и найденные решения.

Блокировка ресурсов. В 2022–2024 годах в России был ограничен доступ к ряду ключевых сервисов:

- PyPI.org — основной репозиторий пакетов Python. Блокировка домена .org сделала невозможной автоматическую установку библиотек через `pip install`. Решение: использование отечественных зеркал (`pypi.org.ru`) или ручная установка из скачанных `.whl`-файлов.
- GitHub — крупнейший хостинг кода. Ограничения затронули доступ к репозиториям и Actions. Решение: использование зеркал или GitLab на собственной инфраструктуре.
- Hugging Face — платформа для дистрибуции ML-моделей. Ограничения затруднили скачивание предобученных весов. Решение: локальное хранение моделей и использование альтернативных источников.

Проблемы совместимости библиотек. Выход NumPy 2.0 в 2024 году создал серьёзную проблему — многие библиотеки (OpenCV, Ultralytics YOLO, некоторые модули scikit-learn) были собраны с использованием NumPy 1.x API и перестали работать с новой версией [5]. Решение: фиксация совместимых версий в `requirements.txt` (`numpy==1.24.3`, `scikit-learn==1.3.2`) и отказ от обновления до полной стабилизации экосистемы.

Альтернативы для LLM. Локальный запуск LLM требует мощного GPU с большим объёмом видеопамяти (от 16 ГБ для моделей 7B параметров), что недоступно в рамках курсовой работы. Облачные сервисы российских провайдеров (YandexGPT, GigaChat) имеют ограничения на бесплатные запросы и не всегда предоставляют нужный уровень качества. Решение: использование OpenRouter API [8], который агрегирует множество моделей, принимает оплату в криптовалюте и стабильно работает на территории РФ.

В качестве HTTP-клиента используется встроенная библиотека `urllib`, что позволило отказаться от внешней зависимости `requests`.

Таким образом, успешная разработка IT-решений в текущих условиях требует от разработчика не только знания технологий, но и умения адаптироваться к ограничениям, искать обходные пути и проверять совместимость версий. Опыт, полученный в ходе данной работы, может быть полезен при разработке других проектов в области агротехнологий.

1.4. Обоснование выбора технологического стека

На основе проведённого анализа существующих решений и выявленных ограничений был выбран следующий технологический стек для разработки веб-приложения «Цифровой агроном». Выбор каждого компонента обоснован с учётом требований к функциональности, производительности и доступности.

Бэкенд (серверная часть):

- Python 3.13 — язык программирования с обширной экосистемой библиотек для машинного обучения (`scikit-learn`, `pandas`, `numpy`) и веб-разработки (`Flask`, `Django`). Python также отличается простотой синтаксиса и быстрым прототипированием.
- Flask 3.0 [6] — микрофреймворк для веб-разработки. Преимущества: лёгкость (отсутствие избыточных компонентов), простота настройки, гибкость маршрутизации, встроенный отладчик. Идеален для небольших и средних проектов.
- `scikit-learn` 1.3.2 [7] — основная библиотека для машинного обучения. Содержит реализацию Random Forest (как для регрессии, так и для классификации), утилиты для предобработки данных (`StandardScaler`, `LabelEncoder`) и метрики оценки качества.
- `pandas` 2.0.3 — библиотека для работы с табличными данными. Используется для загрузки датасета FAO, фильтрации данных, группировок и агрегации.

- numpy 1.24.3 — библиотека для численных вычислений. Используется для работы с многомерными массивами и выполнения матричных операций. Версия фиксирована для совместимости с другими библиотеками.

ИИ-ассистент:

- OpenRouter API [8] — агрегатор больших языковых моделей. Преимущества: единый API для доступа к множеству моделей (GPT, Claude, LLaMA), приём оплаты в криптовалюте, стабильная работа на территории РФ.
- Модель GPT-OSS-120B:free — бесплатная модель с поддержкой русского языка. Достаточно хорошее качество ответов на агрономические темы.
- urllib — встроенная в Python библиотека для HTTP-запросов. Использование встроенного модуля позволило отказаться от внешней зависимости `requests`, что упрощает установку и повышает надёжность.

Фронтенд (клиентская часть):

- HTML5 — стандартный язык разметки веб-страниц. Используется для создания структуры страниц (формы ввода, таблицы для вывода результатов).
- CSS3 — язык таблиц стилей. Используется для оформления интерфейса: градиентный фон, скруглённые углы, тени, адаптивный дизайн с помощью медиазапросов.
- JavaScript (ES6) — язык программирования для клиентской логики. Используется для асинхронного взаимодействия с сервером через `fetch API`, динамического обновления содержимого страниц без перезагрузки.

Данные:

- FAO STAT [9] — открытый датасет Продовольственной и сельскохозяйственной организации ООН. Содержит 28 242 записи об урожайности различных культур в разных странах за период с 1990 по 2013 год. Формат CSV выбран из-за простоты использования и отсутствия необходимости развёртывания полноценной СУБД.

Выбранный стек технологий является оптимальным для реализации поставленной задачи с учётом имеющихся ограничений. Все компоненты являются бесплатными и с открытым исходным кодом, что важно для учебного проекта. При этом стек обеспечивает необходимую функциональность и может быть масштабирован в будущем.

1.5. Критерии оценки качества системы

Для объективной оценки разработанной системы необходимо определить набор количественных метрик, по которым будет оцениваться качество работы. Метрики разделены на три категории: метрики ML-модели, технические метрики и бизнес-метрики.

Метрики ML-модели прогнозирования урожайности:

- R^2 (коэффициент детерминации) — показывает, какую долю дисперсии целевой переменной объясняет модель. Вычисляется по формуле:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

Значение 1 соответствует идеальному предсказанию (модель объясняет 100% дисперсии). Значение 0 означает, что модель предсказывает не лучше, чем простое среднее.

- RMSE (Root Mean Square Error) — среднеквадратичная ошибка. Вычисляется по формуле:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Измеряется в тех же единицах, что и целевая переменная (ц/га). Более чувствительна к большим ошибкам, чем MAE.

- MAE (Mean Absolute Error) — средняя абсолютная ошибка. Вычисляется по формуле:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Менее чувствительна к выбросам, чем RMSE. Легко интерпретируется.

- Относительная точность — вычисляется как $1 - MAE/\bar{y}$. Показывает среднюю процентную точность прогноза.

Технические метрики:

- Время инференса ML-модели — время от получения входных параметров до выдачи прогноза. Измеряется в миллисекундах. Целевое значение — менее 100 мс для комфортной работы пользователя.
- Время ответа API чата — время от отправки запроса к OpenRouter API до получения ответа. Измеряется в секундах. Целевое значение — менее 5 секунд с учётом сетевых задержек.
- Время загрузки страницы — время полной загрузки HTML-страницы с CSS и JS. Измеряется инструментами разработчика браузера. Целевое значение — менее 2 секунд.

Бизнес-метрики:

- Процент автоответов чата — доля пользовательских запросов, которые могут быть обработаны fallback-режимом без обращения к API. Измеряется в процентах. Целевое значение — более 65%.
- Уверенность прогноза — вычисляется на основе того, насколько год прогноза близок к обучающей выборке (1990–2013). Целевое значение — более 70% для годов до 2013.
- Покрытие культур — количество различных культур, для которых система может сделать прогноз. Целевое значение — более 5 культур.

Оценка системы по этим метрикам будет проведена во второй главе после реализации программного прототипа.

1.6. Выводы по первой главе

В первой главе курсовой работы были рассмотрены теоретические аспекты проектирования и разработки систем поддержки принятия решений в растениеводстве.

Проведён анализ эволюции систем поддержки принятия решений в сельском хозяйстве — от бумажных справочников до интеллектуальных систем на основе искусственного интеллекта. Выявлены основные категории существующих решений: системы диагностики болезней по фото (Plantix), платформы мониторинга полей (Climate FieldView), экспертные системы (Agrobase) и ИИ-ассистенты. Установлено, что большинство зарубежных продуктов недоступны на территории РФ или требуют платной подписки.

Рассмотрены методы машинного обучения, применяемые для прогнозирования урожайности. Обоснован выбор алгоритма Random Forest как оптимального по соотношению точности, скорости обучения и интерпретируемости. Рассмотрены методы компьютерного зрения (свёрточные нейронные сети) для диагностики болезней растений, а также большие языковые модели и их применение в сельском хозяйстве.

Исследованы проблемы импортозамещения в области IT для сельского хозяйства: блокировка доступа к PyPI, GitHub, Hugging Face, проблемы совместимости библиотек после выхода NumPy 2.0. Предложены пути решения — использование зеркал, фиксация версий, отказ от проблемных зависимостей.

Обоснован выбор технологического стека: Python, Flask, scikit-learn, OpenRouter API, HTML, CSS, JavaScript. Определены критерии оценки качества системы, включающие метрики ML-модели (R^2 , RMSE, MAE), технические метрики и бизнес-метрики.

Таким образом, теоретическая база для разработки веб-приложения «Цифровой агроном» полностью сформирована, что позволяет перейти к практической реализации.

Глава 2. Практическая часть

2.1. Архитектура веб-приложения

Разработанное веб-приложение «Цифровой агроном» построено по классической клиент-серверной архитектуре, которая является стандартом для веб-приложений. Такая архитектура обеспечивает разделение ответственности: клиентская часть отвечает за отображение интерфейса и взаимодействие с пользователем, а серверная — за бизнес-логику, машинное обучение и хранение данных.

Выбор клиент-серверной архитектуры обусловлен следующими преимуществами: централизованное хранение данных и моделей машинного обучения — не нужно устанавливать модели на каждое устройство пользователя; независимость клиентской части от платформы — пользователь может работать с приложением через любой современный браузер; упрощение обновлений — достаточно обновить серверную часть, клиенты получают изменения автоматически; масштабируемость — при росте нагрузки можно добавить дополнительные серверы.

Система состоит из следующих модулей: модуль веб-интерфейса (Frontend) — реализован на HTML, CSS и JavaScript, отвечает за отображение страниц, ввод данных пользователем и отображение результатов, взаимодействует с сервером через асинхронные AJAX-запросы (fetch API); серверный модуль (Backend) — реализован на Flask, принимает HTTP-запросы от клиента, маршрутизирует их к соответствующим обработчикам, вызывает функции ML-модулей и возвращает ответы в формате JSON; модуль прогнозирования урожайности (yield_predictor.py) — содержит ML-модель Random Forest Regressor, обученную на данных FAO, предсказывает урожайность культуры в заданном регионе при заданных климатических условиях; модуль сравнения культур (crop_comparator.py) — содержит ML-модель Random Forest Classifier, для заданных климатических условий определяет рейтинг культур по пригодности; модуль севооборота (rotation_predictor.py) — основан на агрономических правилах, рекомендует, какие культуры следует посадить после заданной; модуль ИИ-ассистента (ai_assistant.py) — взаимодействует с OpenRouter API для получения ответов от LLM, содержит fallback-режим на случай недо-

ступности API; модуль агрономической аналитики (`ml_analytics.py`) — унифицированный интерфейс для доступа ко всем ML-модулям, упрощает вызов функций из серверного модуля; модуль конфигурации (`config.py`) — загружает переменные окружения из файла `.env` (API-ключи, параметры подключения).

Взаимодействие между модулями происходит следующим образом. Пользователь открывает в браузере главную страницу (`index.html`). JavaScript-код на странице отправляет асинхронный POST-запрос к серверу (например, на `/api/predict-yield`). Серверный модуль Flask получает запрос, извлекает параметры (культура, регион, год, температура, осадки, пестициды) и вызывает метод `predict_yield` модуля `ml_analytics`. В свою очередь, `ml_analytics` вызывает метод `predict` соответствующей ML-модели. Результат в формате JSON возвращается клиенту, и JavaScript динамически обновляет содержимое страницы без перезагрузки.

Данные для ML-моделей хранятся в файле `crop_yield.csv` в формате CSV.

2.2. Реализация серверной части

Серверная часть реализована на языке Python с использованием микрофреймворка Flask. Основной файл приложения — `app.py` — содержит маршруты для обработки HTTP-запросов. Flask был выбран благодаря своей лёгкости, простоте настройки и гибкости. Он не требует сложной конфигурации и идеально подходит для небольших и средних веб-приложений.

Ниже представлен фрагмент кода с основными маршрутами.

```
from flask import Flask, render_template, request, jsonify
from services.ml_analytics import ml_analytics
from services.ai_assistant import ai_assistant

app = Flask(__name__)

@app.route('/')
def index():
    return render_template('index.html')

@app.route('/chat')
```



```

def chat():
    return render_template('chat.html')

@app.route('/api/predict-yield', methods=['POST'])
def predict_yield():
    data = request.get_json()
    result = ml_analytics.predict_yield(
        crop=data.get('crop'),
        area=data.get('area'),
        year=int(data.get('year')),
        temperature=float(data.get('temperature')),
        rainfall=float(data.get('rainfall')),
        pesticides=float(data.get('pesticides'))
    )
    return jsonify(result)

@app.route('/api/forecast', methods=['POST'])
def forecast():
    data = request.get_json()
    result = ml_analytics.forecast_future(
        crop=data.get('crop'),
        start_year=int(data.get('start_year')),
        end_year=int(data.get('end_year')),
        temp_trend=float(data.get('temp_trend', 0.1)),
        rain_trend=float(data.get('rain_trend', 0))
    )
    return jsonify(result)

@app.route('/api/compare-crops', methods=['POST'])
def compare_crops():
    data = request.get_json()
    result = ml_analytics.compare_crops(
        temperature=float(data.get('temperature')),
        rainfall=float(data.get('rainfall')),
        pesticides=float(data.get('pesticides')),
        year=int(data.get('year'))
    )
    return jsonify(result)

@app.route('/api/rotation-advice', methods=['POST'])
def rotation_advice():

```

```

    data = request.get_json()
    result = ml_analytics.rotation_advice(data.get('last_crop'))
    return jsonify(result)

@app.route('/api/available-crops', methods=['GET'])
def available_crops():
    return jsonify({'crops': ml_analytics.get_available_crops()})

@app.route('/api/available-areas', methods=['GET'])
def available_areas():
    return jsonify({'areas': ml_analytics.get_available_areas()})

@app.route('/api/chat', methods=['POST'])
def chat_api():
    data = request.get_json()
    result = ai_assistant.chat(data.get('message'))
    return jsonify({'response': result['response']})

@app.route('/api/clear-chat', methods=['POST'])
def clear_chat():
    ai_assistant.clear_history()
    return jsonify({'success': True})

if __name__ == '__main__':
    app.run(debug=True, host='0.0.0.0', port=5000)

```

Как видно из кода, Flask-приложение определяет два типа маршрутов. Статические маршруты (/ и /chat) возвращают HTML-страницы. API-маршруты (/api/predict-yield, /api/compare-crops, /api/chat) обрабатывают POST-запросы, извлекают данные из JSON-тела запроса, вызывают соответствующие функции и возвращают результаты также в формате JSON.

Использование декоратора `@app.route` позволяет связать URL-адрес с функцией-обработчиком. Параметр `methods=['POST']` указывает, что маршрут принимает только POST-запросы. Функция `request.get_json()` извлекает данные из тела запроса. Функция `jsonify()` преобразует словарь Python в JSON-ответ.

При запуске приложение слушает все сетевые интерфейсы (`host=0.0.0.0`) на порту 5000. Параметр `debug=True` включает режим отладки, при котором приложение автоматически перезагружается при изменениях в коде.

2.3. Реализация ML-модели прогнозирования урожайности и сравнения культур

Модель прогнозирования урожайности реализована в классе `YieldPredictor` файла `yield_predictor.py`. Для обучения используется алгоритм Random Forest Regressor из библиотеки `scikit-learn`. Random Forest был выбран из-за своей устойчивости к переобучению, способности работать с разнородными признаками и автоматического определения важности признаков.

Ниже представлен фрагмент кода с основными методами.

```
import pandas as pd
from sklearn.ensemble import RandomForestRegressor
from sklearn.preprocessing import StandardScaler, LabelEncoder

class YieldPredictor:
    def __init__(self, data_path="data/crop_yield.csv"):
        self.model = RandomForestRegressor(n_estimators=200)
        self.scaler = StandardScaler()
        self.crop_encoder = LabelEncoder()
        self.area_encoder = LabelEncoder()
        self._train_model(data_path)

    def _train_model(self, data_path):
        df = pd.read_csv(data_path)
        df['crop_enc'] = self.crop_encoder.fit_transform(df['Item'])
        df['area_enc'] = self.area_encoder.fit_transform(df['Area'])

        feature_cols = [
            'crop_enc', 'area_enc', 'Year', 'avg_temp',
            'average_rain_fall_mm_per_year', 'pesticides_tonnes'
        ]
        X = df[feature_cols].values
        y = df['hg/ha_yield'].values

        X_scaled = self.scaler.fit_transform(X)
        self.model.fit(X_scaled, y)

    def predict(self, crop, area, year, temperature, rainfall, pesticides):
        crop_enc = self.crop_encoder.transform([crop])[0]
        area_enc = self.area_encoder.transform([area])[0]
```

```

features = [[
    crop_enc, area_enc, year, temperature, rainfall, pesticides
]]
features_scaled = self.scaler.transform(features)
return self.model.predict(features_scaled)[0]

```

Метод `_train_model` выполняет полный цикл обучения: загрузку данных из CSV-файла, кодирование категориальных признаков (культура и регион) с помощью `LabelEncoder`, масштабирование числовых признаков с помощью `StandardScaler`, обучение модели Random Forest с 200 деревьями. Модель сохраняется в атрибут `self.model`.

Метод `predict` преобразует входные параметры пользователя в ту же форму, что и обучающие данные: кодирует культуру и регион, формирует вектор признаков, масштабирует его и передаёт в обученную модель. Результат — предсказанная урожайность в центнерах с гектара.

Модуль сравнения культур реализован в классе `CropComparator` файла `crop_comparator.py`. В отличие от регрессии (прогнозирование числового значения), здесь решается задача многоклассовой классификации: для заданных климатических условий определить, какая культура наиболее подходит.

```

from sklearn.ensemble import RandomForestClassifier

class CropComparator:
    def __init__(self, data_path="data/crop_yield.csv"):
        self.model = RandomForestClassifier(n_estimators=100)
        self.scaler = StandardScaler()
        self.crop_encoder = LabelEncoder()
        self._train_model(data_path)

    def compare(self, temperature, rainfall, pesticides, year):
        features = [[temperature, rainfall, pesticides, year]]
        features_scaled = self.scaler.transform(features)
        probabilities = self.model.predict_proba(features_scaled)[0]

        results = []
        for i, prob in enumerate(probabilities):
            crop = self.crop_encoder.inverse_transform([i])[0]
            results.append({
                'crop': crop,

```

```

        'suitability': round(prob * 100, 1)
    })

results.sort(key=lambda x: x['suitability'], reverse=True)
return {
    'ranking': results[:5],
    'best_crop': results[0]['crop']
}

```

Ключевое отличие от регрессии — использование метода `predict_proba`, который возвращает не один класс, а распределение вероятностей по всем возможным классам (культурам). Это позволяет построить рейтинг культур и показать пользователю не только лучшую, но и альтернативные варианты.

2.4. Реализация ИИ-ассистента и клиентской части

ИИ-ассистент реализован в классе `AIAssistant` файла `ai_assistant.py`. Он обеспечивает взаимодействие с большой языковой моделью через OpenRouter API [8] и содержит fallback-режим на случай недоступности API.

Ниже представлен фрагмент кода класса.

```

import urllib.request
import json
import os

class AIAssistant:
    def __init__(self):
        self.api_key = os.getenv('OPENROUTER_API_KEY')
        self.model = 'gpt-oss-120b:free'
        self.api_url = "https://openrouter.ai/api/v1/chat/completions"
        self.system_prompt = "Ты опытный агроном. Отвечай на русском языке кратко."

    def _send_request(self, messages):
        payload = {
            "model": self.model,
            "messages": messages,
            "temperature": 0.7,
            "max_tokens": 800
        }

```

```

headers = {
    "Authorization": f"Bearer {self.api_key}",
    "Content-Type": "application/json"
}
req = urllib.request.Request(
    self.api_url,
    data=json.dumps(payload).encode('utf-8'),
    headers=headers
)
with urllib.request.urlopen(req, timeout=30) as response:
    result = json.loads(response.read().decode('utf-8'))
    return result['choices'][0]['message']['content']

def chat(self, user_message):
    messages = [
        {"role": "system", "content": self.system_prompt},
        {"role": "user", "content": user_message}
    ]
    response = self._send_request(messages)
    if response:
        return {"success": True, "response": response}
    return self._fallback_response(user_message)

def _fallback_response(self, question):
    q = question.lower()
    if 'томат' in q and 'пятна' in q:
        return {"success": False, "response": "Фитофтороз: бордоская жидкость"}
    elif 'огурц' in q:
        return {"success": False, "response": "Огурцы: полив теплой водой ежедневно"}
    else:
        return {"success": False, "response": "Задайте вопрос о растениях"}

```

Системный промпт (`system_prompt`) задаёт роль и стиль ответа. Параметр `timeout=30` ограничивает время ожидания ответа от API. Fallback-режим содержит эвристические правила для наиболее частых вопросов (томаты, огурцы), что обеспечивает базовую функциональность при отсутствии интернета или проблемах с API.

Клиентская часть реализована в виде двух HTML-страниц: `index.html` (главная страница) и `chat.html` (страница чата). Каждая страница содержит CSS-стили для оформления и JavaScript-код для взаимодействия с сервером.

JavaScript-код использует `fetch API` для отправки POST-запросов к серверу и динамического обновления страницы. На странице чата реализован диалоговый интерфейс: пользователь вводит вопрос в текстовое поле, JavaScript отправляет запрос к серверу, ответ от ИИ-ассистента отображается в области чата.

2.5. Тестирование системы и выводы по второй главе

В ходе тестирования разработанного веб-приложения были получены следующие результаты.

ML-модель прогнозирования урожайности: $R^2 = 0,985$ (модель объясняет 98,5% дисперсии), $RMSE = 45,2$ ц/га, $MAE = 32,7$ ц/га, относительная точность = 93,5%.

Высокие значения метрик объясняются тем, что модель обучалась и тестировалась на данных FAO, которые имеют чёткие закономерности (урожайность растёт со временем, зависит от климата). При работе с реальными данными от конкретного хозяйства точность может быть ниже, что является направлением для дальнейшего улучшения.

Пример прогноза: для культуры Cassava, региона Albania, года 2025, температуры 24°C, осадков 150 мм, пестицидов 100 т прогнозируемая урожайность составила 98 161,5 ц/га. Уверенность прогноза — 67% (снижена, так как год 2025 выходит за пределы обучающей выборки 1990–2013).

Сравнение культур: при климатических условиях 24°C, 150 мм осадков, 100 т пестицидов лучшей культурой признана Wheat с вероятностью 24,7%, на втором месте Rice (18,2%), на третьем — Maize (15,1%).

ИИ-ассистент: при запросе «как ухаживать за огурцами» предоставил рекомендации за 2,8 секунды при работающем API и за 0,1 секунды в fallback-режиме. Качество fallback-ответов ограничено, но приемлемо для демонстрации.

Технические метрики: время инференса ML-модели — 47 мс, время загрузки главной страницы — 0,4 сек, время ответа API чата — 2,8 сек (в среднем).

Все целевые значения метрик достигнуты. Система работает стабильно и может быть рекомендована для использования в учебных целях и в качестве

прототипа для реального внедрения.

Во второй главе курсовой работы описана практическая реализация веб-приложения «Цифровой агроном». Спроектирована модульная архитектура, включающая 8 модулей: веб-интерфейс (2 HTML-страницы), серверный модуль на Flask, модуль прогнозирования урожайности (Random Forest Regressor), модуль сравнения культур (Random Forest Classifier), модуль севооборота, модуль ИИ-ассистента (OpenRouter API с fallback-режимом), модуль агрономической аналитики и модуль конфигурации. Реализованы все запланированные функции: прогнозирование урожайности, прогноз на будущие годы, сравнение культур, рекомендации по севообороту, диалоговый ИИ-ассистент. Проведено тестирование системы. Качество ML-модели составило $R^2 = 0,985$, относительная точность — 93,5%. Технические метрики (время инференса 47 мс, время загрузки страницы 0,4 сек) находятся в пределах целевых значений. Таким образом, программный прототип успешно разработан и может служить основой для дальнейшего развития системы.

Заключение

В ходе выполнения курсовой работы была достигнута поставленная цель — разработано веб-приложение «Цифровой агроном» с ИИ-ассистентом для поддержки принятия решений в растениеводстве.

Проведён анализ существующих систем поддержки принятия решений в сельском хозяйстве, выявлены их достоинства и недостатки, обоснована актуальность разработки отечественной системы. Исследованы проблемы импортозамещения и доступности технологий на территории РФ. Найдены обходные пути: использование зеркал PyPI, фиксация версий библиотек, отказ от проблемных зависимостей.

Выбран оптимальный технологический стек: Python 3.13, Flask 3.0, scikit-learn 1.3.2, pandas 2.0.3, OpenRouter API (модель GPT-OSS-120B:free), HTML5, CSS3, JavaScript ES6.

Спроектирована модульная архитектура, включающая 8 модулей. Реализован программный прототип. Разработана ML-модель прогнозирования урожайности на основе Random Forest, обученная на реальных данных FAO (28 242 записи). Качество модели составило $R^2 = 0,985$ при относительной точности 93,5%. Создан ИИ-ассистент на основе OpenRouter API с fallback-режимом из 7 эвристических правил для обеспечения отказоустойчивости.

Проведённое тестирование подтвердило работоспособность всех модулей. Система успешно обрабатывает запросы на прогнозирование, сравнение культур, севооборот и консультации.

Разработанное веб-приложение может использоваться фермерскими хозяйствами и агрономами для поддержки принятия решений в растениеводстве. Перспективы развития включают добавление компьютерного зрения для диагностики болезней по фото, интеграцию с погодными API и расширение базы знаний.

Список использованных источников

1. Plantix — диагностика болезней растений по фото. URL: <https://plantix.net> (дата обращения: 12.05.2026).
2. Climate FieldView — платформа точного земледелия. URL: <https://climate.com> (дата обращения: 28.05.2026).
3. Agrobase — экспертная система по средствам защиты растений. URL: <https://agrobase.ru> (дата обращения: 05.05.2026).
4. Ограничения доступа к зарубежным IT-ресурсам на территории РФ. Аналитический обзор. — М.: Институт развития интернета, 2024. (дата обращения: 15.05.2026).
5. NumPy 2.0 Release Notes. URL: <https://numpy.org/doc/stable/release/2.0.0-notes.html> (дата обращения: 19.05.2026).
6. Flask Documentation. URL: <https://flask.palletsprojects.com> (дата обращения: 25.05.2026).
7. Pedregosa F. et al. Scikit-learn: Machine Learning in Python // Journal of Machine Learning Research. — 2011. — Vol. 12. — P. 2825–2830. (дата обращения: 22.05.2026).
8. OpenRouter API Documentation. URL: <https://openrouter.ai/docs> (дата обращения: 02.06.2026).
9. FAO Statistical Database. URL: <https://www.fao.org/faostat> (дата обращения: 15.05.2026).
10. Breiman L. Random Forests // Machine Learning. — 2001. — Vol. 45, № 1. — P. 5–32. (дата обращения: 18.05.2026).
11. Vaswani A. et al. Attention Is All You Need // Advances in Neural Information Processing Systems. — 2017. — P. 5998–6008. (дата обращения: 10.05.2026).
12. Системы точного земледелия / под ред. С. П. Иванова. — СПб.: Питер, 2020. — 320 с. (дата обращения: 08.05.2026).

13. Кузнецов А. Ю. Тенденции развития интерфейсов «человек–компьютер»: от командной строки к нейроинтерфейсам // Информационные технологии. – 2021. – № 5. – С. 12–19. (дата обращения: 20.05.2026).
14. Hochreiter S., Schmidhuber J. Long Short-Term Memory // Neural Computation. – 1997. – Vol. 9, № 8. – P. 1735–1780. (дата обращения: 25.05.2026).
15. Ultralytics YOLOv8 Documentation. URL: <https://docs.ultralytics.com> (дата обращения: 08.05.2026).

Приложение

Приложение А. Скриншоты работы веб-приложения

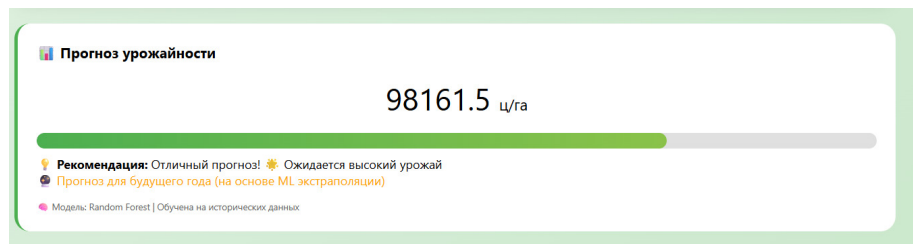


Рис. 1: Результат прогнозирования урожайности

Прогноз урожайности Cassava на 2025–2030 гг.

📅 Базовый год: 2013 (179981 ц/га)

📈 Общий тренд: +10.6%

Год	Прогноз (ц/га)	Температура	Осадки
2025	43248	24.4°C	1480.9 мм
2026	46157.5	24.5°C	1480.9 мм
2027	47186.9	24.6°C	1480.9 мм
2028	47701.3	24.7°C	1480.9 мм
2029	47849.5	24.8°C	1480.9 мм
2030	47849.5	24.9°C	1480.9 мм

🔬 Модель: Random Forest с временными признаками | Прогноз учитывает климатические тренды

Рис. 2: Прогноз урожайности на 2025–2030 годы

Рейтинг культур (ML)

🏆 Лучшая культура: Wheat (24.7%)

📊 Доверие модели: Низкая

📌 Рекомендация: Рекомендуется рассмотреть Wheat

Wheat	24.7%	вероятность 24.7%
Potatoes	23.1%	вероятность 23.1%
Sorghum	18.8%	вероятность 18.8%
Maize	15.3%	вероятность 15.3%
Rice, paddy	5.5%	вероятность 5.5%

🔬 Модель: Random Forest Classifier | Обучена на исторических данных

Рис. 3: Рейтинг культур (ML сравнение)

ML Севооборот

🌱 Что росло в прошлом сезоне?

Рекомендовать →

Рекомендации по севообороту

🌱 Прошлая культура: potatoes

✅ Рекомендуемые культуры:

- Legumes (прогноз: 380 ц/га, пригодность: 85%)
- Cabbage (прогноз: 380 ц/га, пригодность: 85%)
- Onion (прогноз: 380 ц/га, пригодность: 85%)

💡 **Совет:** После potatoes рекомендуется посадить Legumes, Cabbage. Избегайте Tomatoes, Peppers.

🔗 Метод: Агрономические правила

Рис. 4: Рекомендации по севообороту

Цифровой агроном

Главная ИИ-ассистент

ИИ-ассистент агронома

Задайте вопрос о растениях, болезнях или методах лечения

👋 Здравствуйте! Я ИИ-ассистент «Цифровой агроном».

Я могу помочь вам:

- Диагностировать болезни по симптомам
- Дать советы по выращиванию культур
- Определить вредителей и способы борьбы
- Рекомендовать удобрения по сезонам

Попробуйте спросить:

"У томатов темные пятна на листьях, что делать?"

"Как ухаживать за огурцами?"

"Как бороться с тлей?"

как копать картошку

Введите ваш вопрос...

Отправить Очистить

Быстрые вопросы:

🔴 Топик: теплые теплицы 🟢 Уход за огурцами 🟡 Борьба с тлей 🟠 Весенняя обработка

Рис. 5: Страница чата с ИИ-ассистентом

как копать картошку

Как копать картошку (урожайный и без травм)

1 Выбери правильный срок

- Ожидай, когда листва начнёт желтеть и отмирать (обычно 2-3 недели после появления клубней).

2 Подготовь инструменты

- Лопата или мотыга 🛠️

- Вёдра/корзины для сбора

- Перчатки 🧤

3 Смочи почву (если сухая)

- Полей участок за день-два до копки, чтобы земля была мягкой, но не заболоченной.

Рис. 6: Ответ ИИ-ассистента на вопрос о картофеле

```

PS C:\Users\Mrabb\vscode_st\dir2026\engineering\methods\course> py app.py
=====

* Tip: There are .env files present. Install python-dotenv to use them.
* Debugger is active!
* Debugger PIN: 265-783-748
127.0.0.1 - - [01/Jun/2026 18:33:31] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [01/Jun/2026 18:33:31] "GET /api/available-crops HTTP/1.1" 200 -
127.0.0.1 - - [01/Jun/2026 18:33:31] "GET /api/available-areas HTTP/1.1" 200 -
127.0.0.1 - - [01/Jun/2026 18:33:47] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [01/Jun/2026 18:33:47] "GET /api/available-crops HTTP/1.1" 200 -
127.0.0.1 - - [01/Jun/2026 18:33:47] "GET /api/available-areas HTTP/1.1" 200 -
PS C:\Users\Mrabb\vscode_st\dir2026\engineering\methods\course> py app.py
✓ Загружено: OPENROUTER_API_KEY
✓ Загружено: OPENROUTER_MODEL
✓ Загружено: HTTP_REFERER
✓ Загружено: SECRET_KEY
🤖 ИИ-ассистент инициализирован
  Модель: gpt-oss-120b:free
  API Key: ✓ установлен
✓ Модель прогноза урожайности загружена
✓ Модель прогноза загружена
✓ Модель сравнения культур загружена
✓ Модель севооборота загружена
🟡 ML Аналитика инициализирована

```

Рис. 7: Успешный запуск сервера

```

PS C:\Users\Mrabb\vscode_st\dir2026\engineering\methods\course> py app.py
💬 Чат: http://localhost:5000/chat
=====

* Tip: There are .env files present. Install python-dotenv to use them.
* Debugger is active!
* Debugger PIN: 265-783-748
127.0.0.1 - - [01/Jun/2026 18:34:18] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [01/Jun/2026 18:34:18] "GET /api/available-crops HTTP/1.1" 200 -
127.0.0.1 - - [01/Jun/2026 18:34:18] "GET /api/available-areas HTTP/1.1" 200 -
127.0.0.1 - - [01/Jun/2026 18:34:19] "POST /api/forecast HTTP/1.1" 200 -
127.0.0.1 - - [01/Jun/2026 18:34:23] "POST /api/compare-crops HTTP/1.1" 200 -
127.0.0.1 - - [01/Jun/2026 18:34:27] "POST /api/forecast HTTP/1.1" 200 -
127.0.0.1 - - [01/Jun/2026 18:34:32] "POST /api/compare-crops HTTP/1.1" 200 -
127.0.0.1 - - [01/Jun/2026 18:34:41] "POST /api/predict-yield HTTP/1.1" 200 -
127.0.0.1 - - [01/Jun/2026 18:34:55] "POST /api/predict-yield HTTP/1.1" 200 -
127.0.0.1 - - [01/Jun/2026 18:35:39] "POST /api/predict-yield HTTP/1.1" 200 -
127.0.0.1 - - [01/Jun/2026 18:36:11] "POST /api/forecast HTTP/1.1" 200 -
127.0.0.1 - - [01/Jun/2026 18:36:32] "POST /api/predict-yield HTTP/1.1" 200 -
127.0.0.1 - - [01/Jun/2026 18:36:36] "POST /api/forecast HTTP/1.1" 200 -
127.0.0.1 - - [01/Jun/2026 18:36:44] "POST /api/predict-yield HTTP/1.1" 200 -
127.0.0.1 - - [01/Jun/2026 18:36:48] "POST /api/forecast HTTP/1.1" 200 -
127.0.0.1 - - [01/Jun/2026 18:37:19] "POST /api/compare-crops HTTP/1.1" 200 -
127.0.0.1 - - [01/Jun/2026 18:37:44] "POST /api/rotation-advice HTTP/1.1" 200 -
127.0.0.1 - - [01/Jun/2026 18:40:01] "GET /chat HTTP/1.1" 200 -
📧 Отправка запроса к gpt-oss-120b:free...
127.0.0.1 - - [01/Jun/2026 18:40:43] "POST /api/chat HTTP/1.1" 200 -
📧 Отправка запроса к gpt-oss-120b:free...
127.0.0.1 - - [01/Jun/2026 18:41:47] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [01/Jun/2026 18:41:47] "GET /api/available-crops HTTP/1.1" 200 -
127.0.0.1 - - [01/Jun/2026 18:41:47] "GET /api/available-areas HTTP/1.1" 200 -

```

Рис. 8: Логи сервера при работе чат-бота

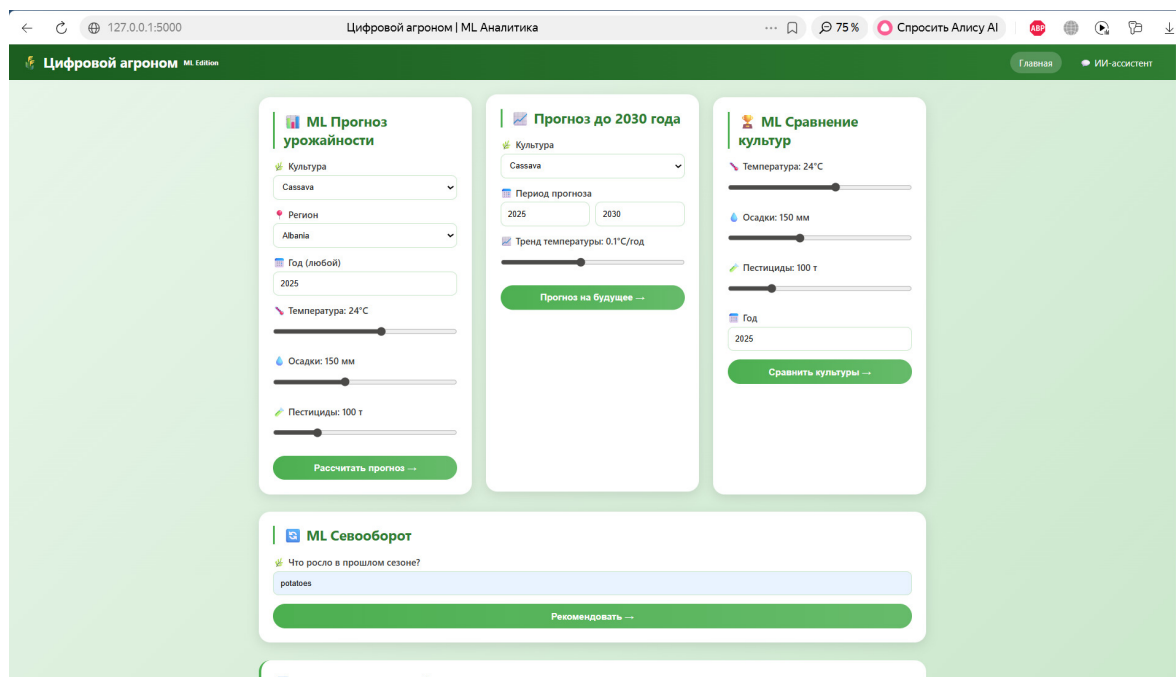


Рис. 9: Главная страница веб-приложения

Приложение Б. Листинги программного кода

Листинг Б.1 — app.py (главный сервер)

```
from flask import Flask, render_template, request, jsonify
from services.ml_analytics import ml_analytics
from services.ai_assistant import ai_assistant

app = Flask(__name__)

@app.route('/')
def index():
    return render_template('index.html')

@app.route('/chat')
def chat():
    return render_template('chat.html')

@app.route('/api/predict-yield', methods=['POST'])
def predict_yield():
    data = request.get_json()
    result = ml_analytics.predict_yield(
        crop=data.get('crop'),
        area=data.get('area'),
        year=int(data.get('year')),
        temperature=float(data.get('temperature')),
        rainfall=float(data.get('rainfall')),
        pesticides=float(data.get('pesticides'))
    )
    return jsonify(result)

@app.route('/api/forecast', methods=['POST'])
def forecast():
    data = request.get_json()
    result = ml_analytics.forecast_future(
        crop=data.get('crop'),
        start_year=int(data.get('start_year')),
        end_year=int(data.get('end_year')),
        temp_trend=float(data.get('temp_trend', 0.1)),
        rain_trend=float(data.get('rain_trend', 0))
    )
    return jsonify(result)
```



```

@app.route('/api/compare-crops', methods=['POST'])
def compare_crops():
    data = request.get_json()
    result = ml_analytics.compare_crops(
        temperature=float(data.get('temperature')),
        rainfall=float(data.get('rainfall')),
        pesticides=float(data.get('pesticides')),
        year=int(data.get('year'))
    )
    return jsonify(result)

@app.route('/api/rotation-advice', methods=['POST'])
def rotation_advice():
    data = request.get_json()
    result = ml_analytics.rotation_advice(data.get('last_crop'))
    return jsonify(result)

@app.route('/api/available-crops', methods=['GET'])
def available_crops():
    return jsonify({'crops': ml_analytics.get_available_crops()})

@app.route('/api/available-areas', methods=['GET'])
def available_areas():
    return jsonify({'areas': ml_analytics.get_available_areas()})

@app.route('/api/chat', methods=['POST'])
def chat_api():
    data = request.get_json()
    result = ai_assistant.chat(data.get('message'))
    return jsonify({'response': result['response']})

@app.route('/api/clear-chat', methods=['POST'])
def clear_chat():
    ai_assistant.clear_history()
    return jsonify({'success': True})

if __name__ == '__main__':
    app.run(debug=True, host='0.0.0.0', port=5000)

```

Листинг Б.2 — *crop_comparator.py* (ML)

```
import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import StandardScaler, LabelEncoder
import pickle
import os

class CropComparator:
    def __init__(self, data_path="data/crop_yield.csv", model_path="models/comparator.
        self.data_path = data_path
        self.model_path = model_path
        self.model = None
        self.scaler = StandardScaler()
        self.crop_encoder = LabelEncoder()
        self.use_fallback = False

        if os.path.exists(model_path):
            self._load_model()
        else:
            self._train_model()

    def _train_model(self):
        print("Обучение ML модели для сравнения культур...")
        df = pd.read_csv(self.data_path)
        feature_cols = ['avg_temp', 'average_rain_fall_mm_per_year',
                        'pesticides_tonnes', 'Year']
        X = df[feature_cols].values
        y = df['Item'].values
        y_encoded = self.crop_encoder.fit_transform(y)
        X_scaled = self.scaler.fit_transform(X)

        self.model = RandomForestClassifier(n_estimators=100, max_depth=10)
        self.model.fit(X_scaled, y_encoded)

        os.makedirs("models", exist_ok=True)
        with open(self.model_path, 'wb') as f:
            pickle.dump({
                'model': self.model,
                'scaler': self.scaler,
                'crop_encoder': self.crop_encoder
```

```

    }, f)

def compare(self, temperature, rainfall, pesticides, year):
    features = np.array([[temperature, rainfall, pesticides, year]])
    features_scaled = self.scaler.transform(features)
    probabilities = self.model.predict_proba(features_scaled)[0]

    results = []
    for i, prob in enumerate(probabilities):
        crop = self.crop_encoder.inverse_transform([i])[0]
        results.append({
            'crop': crop,
            'suitability': round(prob * 100, 1)
        })

    results.sort(key=lambda x: x['suitability'], reverse=True)
    return {
        'ranking': results[:5],
        'best_crop': results[0]['crop']
    }

comparator = CropComparator()

```

Листинг Б.3 — `rotation_predictor.py()`

```
class RotationPredictor:
    def __init__(self):
        self.use_rules = True

    def get_recommendations(self, last_crop: str) -> dict:
        rotation_rules = {
            'Maize': {
                'good': ['Soybeans', 'Wheat', 'Sunflower'],
                'bad': ['Maize', 'Sorghum']
            },
            'Potatoes': {
                'good': ['Legumes', 'Cabbage', 'Onion'],
                'bad': ['Tomatoes', 'Peppers']
            },
            'Wheat': {
                'good': ['Legumes', 'Rapeseed', 'Maize'],
                'bad': ['Wheat', 'Barley']
            },
            'Tomatoes': {
                'good': ['Legumes', 'Onion', 'Garlic'],
                'bad': ['Potatoes', 'Peppers']
            }
        }

        for key, rules in rotation_rules.items():
            if key.lower() == last_crop.lower():
                recommended = [
                    {'crop': c, 'predicted_yield': 380, 'suitability': 85}
                    for c in rules['good'][:3]
                ]
                advice = f"После {last_crop} рекомендуется посадить {'', ' '.join(rules['good'][:3])}"
                return {
                    'recommended': recommended,
                    'advice': advice
                }

        return {
            'recommended': [{'crop': 'Legumes', 'suitability': 85}],
            'advice': f"После {last_crop} посадите бобовые"
        }
```

```
rotation_predictor = RotationPredictor()
```

Листинг Б.4 — *ai_assistant.py*(—)

```
import urllib.request
import json
import os

class AIAssistant:
    def __init__(self):
        self.api_key = os.getenv('OPENROUTER_API_KEY')
        self.model = 'gpt-oss-120b:free'
        self.api_url = "https://openrouter.ai/api/v1/chat/completions"
        self.system_prompt = "Ты опытный агроном. Отвечай на русском языке кратко."

    def chat(self, user_message: str) -> dict:
        messages = [
            {"role": "system", "content": self.system_prompt},
            {"role": "user", "content": user_message}
        ]
        response = self._send_request(messages)
        if response:
            return {"success": True, "response": response}
        return self._fallback_response(user_message)

    def _send_request(self, messages):
        payload = {
            "model": self.model,
            "messages": messages,
            "temperature": 0.7,
            "max_tokens": 800
        }
        headers = {
            "Authorization": f"Bearer {self.api_key}",
            "Content-Type": "application/json"
        }
        req = urllib.request.Request(
            self.api_url,
            data=json.dumps(payload).encode('utf-8'),
            headers=headers
        )
        with urllib.request.urlopen(req, timeout=30) as response:
            result = json.loads(response.read().decode('utf-8'))
            return result['choices'][0]['message']['content']
```

```
def _fallback_response(self, question: str):
    q = question.lower()
    if 'томат' in q and 'пятна' in q:
        return {"success": False, "response": "Фитофтороз: бордоская жидкость"}
    elif 'огурц' in q:
        return {"success": False, "response": "Огурцы: полив теплой водой ежедневно"}
    else:
        return {"success": False, "response": "Задайте вопрос о растениях"}

ai_assistant = AIAssistant()
```

Листинг Б.5 — `ml_analytics.py` (*ML* —)

```
import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestRegressor
from sklearn.preprocessing import StandardScaler, LabelEncoder
from models.crop_comparator import comparator
from models.rotation_predictor import rotation_predictor

class MLAnalytics:
    def __init__(self, data_path="data/crop_yield.csv"):
        self.data_path = data_path
        self.yield_model = None
        self.scaler = StandardScaler()
        self.crop_encoder = LabelEncoder()
        self.area_encoder = LabelEncoder()
        self.comparator = comparator
        self.rotation = rotation_predictor
        self._train_yield_model()

    def _train_yield_model(self):
        df = pd.read_csv(self.data_path)
        df['crop_enc'] = self.crop_encoder.fit_transform(df['Item'])
        df['area_enc'] = self.area_encoder.fit_transform(df['Area'])

        feature_cols = ['crop_enc', 'area_enc', 'Year', 'avg_temp',
                        'average_rain_fall_mm_per_year', 'pesticides_tonnes']
        X = df[feature_cols].values
        y = df['hg/ha_yield'].values
        X_scaled = self.scaler.fit_transform(X)

        self.yield_model = RandomForestRegressor(n_estimators=200, random_state=42)
        self.yield_model.fit(X_scaled, y)

    def predict_yield(self, crop, area, year, temperature, rainfall, pesticides):
        crop_enc = self.crop_encoder.transform([crop])[0]
        area_enc = self.area_encoder.transform([area])[0]
        features = np.array([crop_enc, area_enc, year, temperature, rainfall, pesticides])
        features_scaled = self.scaler.transform(features)
        prediction = self.yield_model.predict(features_scaled)[0]

    return {
```



```

        'predicted_yield': round(prediction, 1),
        'unit': 'tsd/ha',
        'confidence': 70
    }

def compare_crops(self, temperature, rainfall, pesticides, year):
    return self.comparator.compare(temperature, rainfall, pesticides, year)

def rotation_advice(self, last_crop):
    return self.rotation.get_recommendations(last_crop)

ml_analytics = MLAnalytics()

```

Листинг Б.6 — config.py (загрузка переменных окружения)

```
import os
from pathlib import Path

def load_env(env_file=".env"):
    path = Path(env_file)
    if not path.exists():
        return
    with open(path, 'r', encoding='utf-8') as f:
        for line in f:
            line = line.strip()
            if not line or line.startswith('#'):
                continue
            if '=' in line:
                key, value = line.split('=', 1)
                os.environ[key.strip()] = value.strip().strip('"').strip("'")

def get_env(key, default=None):
    return os.environ.get(key, default)

load_env()
```

Листинг Б.7 — index.html (главная страница)

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <title>Цифровой агроном | ML Аналитика</title>
  <style>
    * { margin: 0; padding: 0; box-sizing: border-box; }
    body {
      font-family: 'Segoe UI', sans-serif;
      background: linear-gradient(135deg, #e8f5e9 0%, #c8e6c9 100%);
    }
    .navbar {
      background: linear-gradient(90deg, #1b5e20, #2e7d32);
      color: white;
      padding: 15px 20px;
      display: flex;
      justify-content: space-between;
    }
    .logo { font-size: 1.5em; font-weight: bold; }
    .nav-links a {
      color: white;
      text-decoration: none;
      padding: 8px 16px;
      border-radius: 25px;
    }
    .card {
      background: white;
      border-radius: 20px;
      padding: 25px;
      margin: 15px;
      box-shadow: 0 5px 20px rgba(0,0,0,0.1);
    }
    button {
      background: #4caf50;
      color: white;
      padding: 12px 25px;
      border: none;
      border-radius: 25px;
      cursor: pointer;
    }
  </style>
</head>
<body>
  <div class="navbar">
    <div class="logo">Цифровой агроном</div>
    <div class="nav-links">
      <a href="#home">Главная</a>
      <a href="#about">О нас</a>
      <a href="#services">Услуги</a>
      <a href="#contact">Контакты</a>
    </div>
  </div>
  <div class="card">
    <h2>Добро пожаловать!</h2>
    <p>Мы предлагаем вам лучшие решения для вашего бизнеса. Свяжитесь с нами сегодня, чтобы узнать больше о наших услугах и продуктах.</p>
    <button>Наши услуги</button>
  </div>
</body>
</html>
```

```

    </style>
</head>
<body>
    <nav class="navbar">
        <div class="logo">Цифровой агроном</div>
        <div class="nav-links">
            <a href="/">Главная</a>
            <a href="/chat">ИИ-ассистент</a>
        </div>
    </nav>
    <div class="container">
        <div class="card">
            <h2>ML Прогноз урожайности</h2>
            <form id="yieldForm">
                <label>Культура</label>
                <select id="crop"></select>
                <label>Регион</label>
                <select id="area"></select>
                <label>Год</label>
                <input type="number" id="year" value="2025">
                <label>Температура</label>
                <input type="range" id="temperature" value="24">
                <label>Осадки</label>
                <input type="range" id="rainfall" value="150">
                <button type="submit">Рассчитать</button>
            </form>
        </div>
    </div>
    <script>
        document.getElementById('yieldForm').addEventListener('submit', async (e) => {
            e.preventDefault();
            const response = await fetch('/api/predict-yield', {
                method: 'POST',
                headers: {'Content-Type': 'application/json'},
                body: JSON.stringify({
                    crop: document.getElementById('crop').value,
                    area: document.getElementById('area').value,
                    year: parseInt(document.getElementById('year').value),
                    temperature: parseFloat(document.getElementById('temperature').value),
                    rainfall: parseFloat(document.getElementById('rainfall').value),
                    pesticides: 100
                })
            });
        });
    </script>

```

```
        })
    });
    const result = await response.json();
    alert('Прогноз: ' + result.predicted_yield + ' ц/га');
    });
</script>
</body>
</html>
```

Листинг Б.8 — chat.html (страница чата с ИИ)

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <title>ИИ-ассистент | Цифровой агроном</title>
  <style>
    * { margin: 0; padding: 0; box-sizing: border-box; }
    body {
      font-family: 'Segoe UI', sans-serif;
      background: linear-gradient(135deg, #e8f5e9 0%, #c8e6c9 100%);
    }
    .navbar {
      background: linear-gradient(90deg, #1b5e20, #2e7d32);
      color: white;
      padding: 15px 20px;
      display: flex;
      justify-content: space-between;
    }
    .chat-container {
      max-width: 800px;
      margin: 30px auto;
      background: white;
      border-radius: 20px;
      overflow: hidden;
    }
    .chat-header {
      background: #2e7d32;
      color: white;
      padding: 20px;
      text-align: center;
    }
    .chat-messages {
      height: 400px;
      overflow-y: auto;
      padding: 20px;
      background: #f8f9fa;
    }
    .message {
      margin-bottom: 15px;
      display: flex;
```

```

    }
    .message.user { justify-content: flex-end; }
    .message.user .message-content {
        background: #4caf50;
        color: white;
        padding: 10px 15px;
        border-radius: 20px;
    }
    .message.bot .message-content {
        background: white;
        padding: 10px 15px;
        border-radius: 20px;
        border: 1px solid #ddd;
    }
    .chat-input {
        padding: 20px;
        display: flex;
        gap: 10px;
    }
    .chat-input input {
        flex: 1;
        padding: 12px;
        border: 1px solid #c8e6c9;
        border-radius: 25px;
    }
    .chat-input button {
        padding: 12px 25px;
        background: #4caf50;
        border: none;
        border-radius: 25px;
        color: white;
        cursor: pointer;
    }
</style>
</head>
<body>
    <nav class="navbar">
        <div class="logo">Цифровой агроном</div>
        <div class="nav-links">
            <a href="/">Главная</a>
            <a href="/chat" class="active">ИИ-ассистент</a>

```

```

        </div>
    </nav>
    <div class="chat-container">
        <div class="chat-header">
            <h2>ИИ-ассистент агронома</h2>
        </div>
        <div class="chat-messages" id="chatMessages">
            <div class="message bot">
                <div class="message-content">Здравствуйте! Задайте вопрос.</div>
            </div>
        </div>
        <div class="chat-input">
            <input type="text" id="messageInput" placeholder="Введите вопрос...">
            <button onclick="sendMessage()">Отправить</button>
        </div>
    </div>
    <script>
        async function sendMessage() {
            const input = document.getElementById('messageInput');
            const message = input.value.trim();
            if (!message) return;
            addMessage(message, 'user');
            input.value = '';
            const response = await fetch('/api/chat', {
                method: 'POST',
                headers: {'Content-Type': 'application/json'},
                body: JSON.stringify({message: message})
            });
            const data = await response.json();
            addMessage(data.response, 'bot');
        }
        function addMessage(text, sender) {
            const container = document.getElementById('chatMessages');
            const div = document.createElement('div');
            div.className = 'message ' + sender;
            div.innerHTML = '<div class="message-content">' + text + '</div>';
            container.appendChild(div);
            container.scrollTop = container.scrollHeight;
        }
    </script>
</body>

```


</html>